

Documentação do Projeto: Controle de Reembolso (Frontend)

Este documento descreve a arquitetura, as tecnologias e as decisões de projeto implementadas no frontend do sistema de Controle de Reembolso. Ele foi elaborado para servir como guia de estudo e preparação para entrevistas técnicas.

1. Estrutura Geral do Sistema

O sistema é uma aplicação **SPA (Single Page Application)** moderna, integrada a uma API RESTful.

- **Frontend:** Desenvolvido em React, responsável pela interface do usuário, gestão de estado global, validações de formulários e comunicação com o backend.
 - **Backend (Contexto):** Node.js (conforme documentação do backend), fornecendo endpoints para autenticação, gestão de usuários, categorias e reembolsos.
 - **Comunicação:** Realizada via protocolo HTTP utilizando a biblioteca **Axios**, com autenticação baseada em **JWT (JSON Web Token)**.
-

2. Principais Tecnologias Utilizadas

- **React 19:** Biblioteca principal para construção da interface.
 - **Vite:** Ferramenta de build extremamente rápida que substitui o Create React App.
 - **TypeScript:** Adiciona tipagem estática ao JavaScript, garantindo maior segurança e previsibilidade no código.
 - **Tailwind CSS 4:** Framework utilitário para estilização rápida e responsiva.
 - **React Router Dom 7:** Gerenciamento de rotas e navegação.
 - **React Hook Form + Zod:** Combo para gestão de formulários e validações de esquema rigorosas.
 - **Axios:** Cliente HTTP para chamadas à API.
 - **Vitest + React Testing Library:** Framework de testes unitários e de integração.
-

3. Regras de Negócio (Frontend)

O frontend implementa diversas regras de negócio para garantir a integridade dos dados e uma boa experiência do usuário:

- **Controle de Acesso (RBAC):**
 - **ADMIN:** Visualiza e gerencia usuários e categorias, além de ver todos os reembolsos.
 - **COLABORADOR:** Visualiza apenas seus próprios reembolsos e cria novas solicitações.
 - **Fluxo de Reembolso:**
 - Uma solicitação deve obrigatoriamente ter uma categoria, descrição (min. 3 caracteres), valor (> 0) e data.
 - Anexos (comprovantes) são opcionais, mas integrados ao fluxo de criação/edição.
 - **Persistência de Sessão:** O token JWT é armazenado no `localStorage`, permitindo que o usuário permaneça logado após o refresh da página.
-

4. Funcionamento do Frontend

O sistema está dividido em módulos principais:

1. **Autenticação:** Tela de login com validação de credenciais.
 2. **Dashboard:** Listagem central de reembolsos com filtros por status e busca por descrição.
 3. **Gestão de Categorias:** (Exclusivo Admin) CRUD de categorias que classificam os gastos.
 4. **Gestão de Usuários:** (Exclusivo Admin) Controle de usuários do sistema.
 5. **Solicitação de Reembolso:** Formulário dinâmico para envio de novas despesas.
 6. **Detalhes e Histórico:** Visualização detalhada de um reembolso, incluindo logs de alteração de status.
-

5. Comunicação entre as Partes

A comunicação ocorre de forma centralizada:

- **Instância do Axios:** Configurada em `src/api/api.ts`, onde a `baseUrl` é definida via variáveis de ambiente.
- **Interceptação de Token:** O `AuthContext` injeta automaticamente o `Bearer Token` em todas as requisições após o login.
- **Tratamento de Erros:** Erros da API são capturados e formatados por utilitários (ex: `getApiErrorMessage.ts`) para exibição.

amigável via toasts ou mensagens em tela.

6. Análise da Estrutura de Pastas

```
frontend/
├── src/
│   ├── api/           # Configuração do Axios e serviços.
│   ├── components/    # Componentes reutilizáveis (botões, modais, tabelas).
│   │   ├── categories/
│   │   ├── reimbursement/
│   │   └── users/
│   ├── contexts/      # Contextos globais (Autenticação, Temas).
│   ├── pages/         # Telas principais da aplicação.
│   ├── routes/        # Definição das rotas públicas e protegidas.
│   ├── tests/         # Testes automatizados (unitários e integração).
│   ├── types/         # Interfaces e Tipos TypeScript.
│   ├── utils/         # Funções auxiliares (formatação de moeda, datas).
│   ├── App.tsx        # Componente raiz com Providers e Router.
│   └── main.tsx       # Ponto de entrada do React.
```

- **Conexão:** As **Pages** utilizam os **Components** para construir a interface; os **Contexts** proveem dados globais (como usuário logado) para as **Pages**; e as **Pages** chamam a **API** para buscar/enviar dados.

7. Testes Automatizados

Ferramentas

- **Vitest:** Executor de testes moderno e rápido.
- **React Testing Library (RTL):** Foca em testar o comportamento do componente da perspectiva do usuário.
- **JSDOM:** Simula o ambiente do navegador em Node.js.

O que está sendo testado?

- **Fluxos Críticos:** Login e Criação de Reembolso.
- **Validações:** Se as mensagens de erro aparecem corretamente ao preencher campos inválidos.
- **Integração (Mockada):** As chamadas de API são simuladas (mock) para testar o comportamento do front sem depender do back.

Possíveis Melhorias

- Aumentar a cobertura para componentes menores (UI Library).
- Adicionar testes de ponta a ponta (E2E) com **Playwright** ou **Cypress** para validar o fluxo real entre front e back.

8. Desenvolvimento Assistido por IA

O projeto utilizou o estado da arte em IAs generativas para acelerar e qualificar o desenvolvimento:

- **ChatGPT e Gemini:** Utilizados na fase inicial para **organização e estruturação** do projeto, definição de requisitos e brainstorming da arquitetura.
- **Claude e Antigravity:** Foram os principais motores no **desenvolvimento de código**, auxiliando na escrita de lógica complexa, refatoração e implementação de padrões de projeto.
- **Antigravity e Bananas:** Focaram no **Layout e Interface (UI/UX)**, gerando componentes visuais modernos, escolhendo paletas de cores e garantindo a responsividade.
- **Testes:** IAs foram utilizadas para sugerir casos de teste (edge cases) e gerar esqueletos de testes unitários, garantindo que fluxos de erro fossem devidamente validados.

9. Decisões de Projeto: Validações e Tratamentos

Por que Zod?

Optou-se pelo **Zod** para garantir que os dados que entram no frontend (via formulários) estejam rigorosamente no formato esperado. Isso evita erros de runtime "undefined" e garante que o backend receba dados limpos.

Tratamento de Estados de Carregamento

Utilizamos estados de `loading` no `AuthContext` e nas páginas para evitar que o usuário veja flashes de conteúdo não autenticado ou telas vazias enquanto os dados são buscados.

Feedback de Erro

Todas as chamadas assíncronas são envoltas em blocos `try/catch`. Em caso de erro, o sistema identifica se é um erro de rede, erro de validação do back ou erro inesperado, apresentando uma resposta específica.

10. Passo a Passo da Criação

1. **Setup Inicial:** Inicialização com Vite e configuração do TypeScript/Tailwind.
2. **Arquitetura de Rotas:** Definição de quais páginas seriam públicas (Login) e quais seriam protegidas.
3. **Camada de Autenticação:** Implementação do Context API para gerir o estado de login.
4. **Desenvolvimento das Funcionalidades:** Criação gradual de CRUDs, começando por Categorias, depois Usuários e por fim Reembolsos.
5. **Refinação Visual:** Aplicação de componentes premium (ShadcnUI patterns) e micro-interações.
6. **Ciclo de Testes:** Escrita de testes para garantir que as novas melhorias não quebrassem as funcionalidades existentes.

Documento gerado para apoio técnico e revisão de projeto.